

REPORTE DE PRÁCTICA NO. 8

Análisis sintáctico de sentencias SQL

ALUMNO:

Contreras Hernandez Diego Adonay
Martinez Delgado Baruchs Jesua
Vite Juares Alberick
Navarrete Torres Oscar

PROFESOR: Dr. Eduardo Cornejo-Velázquez



1. Introducción

El diseño e implementación de compiladores es una de las áreas más fundamentales y complejas dentro de las ciencias de la computación. El proceso de compilación se divide tradicionalmente en dos etapas principales: el front-end (análisis) y el back-end (síntesis). Dentro de la fase de análisis, el análisis sintáctico desempeña un papel crucial al recibir la secuencia de componentes léxicos (tokens) generados por el analizador léxico y verificar que dicha secuencia cumpla con las reglas gramaticales del lenguaje de programación (Aho, Sethi y Ullman, 1998). En lenguajes basados en gramáticas formales, el análisis sintáctico resulta esencial para facilitar la comprensión de la estructura del código fuente y abordar de manera efectiva errores difíciles de identificar, como la ambigüedad en la formulación de ciertas sentencias. Esta práctica se centra en la construcción de un analizador sintáctico para una de las estructuras de control de flujo más importantes: el condicional if-then. Utilizando herramientas estándar de la industria como Lex (para el análisis léxico) y Yacc (para el análisis sintáctico), se demostrará cómo una gramática de contexto libre puede ser automatizada para validar estructuras complejas de manera eficiente.

2. Marco teórico

El desarrollo de un traductor o compilador se divide en varias fases secuenciales. Esta práctica se centra en las dos primeras: el análisis léxico y el análisis sintáctico.

1. **Análisis Léxico (Escaneo)** Es la primera fase de un compilador. Lee el flujo de caracteres de entrada y los agrupa en unidades con significado propio llamadas tokens.

Tokens: Son las categorías gramaticales del lenguaje (SUMA, ID, ENTERO).

Lexema: Es la secuencia de caracteres real que coinciden con un token ("3.1416" es el lexema para el token DECIMAL).

Autómatas Finitos Deterministas (AFD): Son modelos matemáticos que sirven para reconocer lenguajes regulares. Flex utiliza estos autómatas para identificar patrones definidos mediante expresiones regulares.

2. **Análisis Sintáctico (Parsing)** Esta fase recibe los tokens generados por el analizador léxico y determina si la estructura de la oración u operación es válida según las reglas gramaticales.

Gramáticas Libres de Contexto (GLC): Son un conjunto de reglas que definen cómo se pueden combinar los tokens. Se representan comúnmente mediante la notación BNF (Backus-Naur Form).

Jerarquía de Operadores: Para que una calculadora funcione correctamente, el analizador debe aplicar reglas de precedencia. En este proyecto, se implementa una estructura jerárquica donde la multiplicación y división tienen mayor prioridad que la suma y resta.

Árbol de Análisis Sintáctico: Es una representación jerárquica que muestra cómo una cadena de entrada se deriva de las reglas de la gramática.

4. **Manejo de Errores** Un compilador robusto debe ser capaz de identificar y reportar errores en dos niveles:
Error Léxico: Ocurre cuando el escáner encuentra un carácter que no pertenece a ningún token válido (ej. un símbolo @ en una expresión aritmética).

Error Sintáctico: Ocurre cuando los tokens son válidos pero su orden no respeta las reglas de la gramática (ej. $5 + * 3$).

3. Herramientas empleadas

(Flex y Bison) Flex (Fast Lexical Analyzer): Es una herramienta para generar escáneres. Su entrada es un archivo .l que contiene expresiones regulares y su salida es código fuente en C que implementa el autómata.

Bison (Yacc compatible): Es un generador de analizadores sintácticos de propósito general. Convierte una gramática libre de contexto en un analizador LALR (Look-Ahead Left-to-Right). Utiliza una pila para procesar los tokens y realizar reducciones gramaticales.

4. Objetivos

general

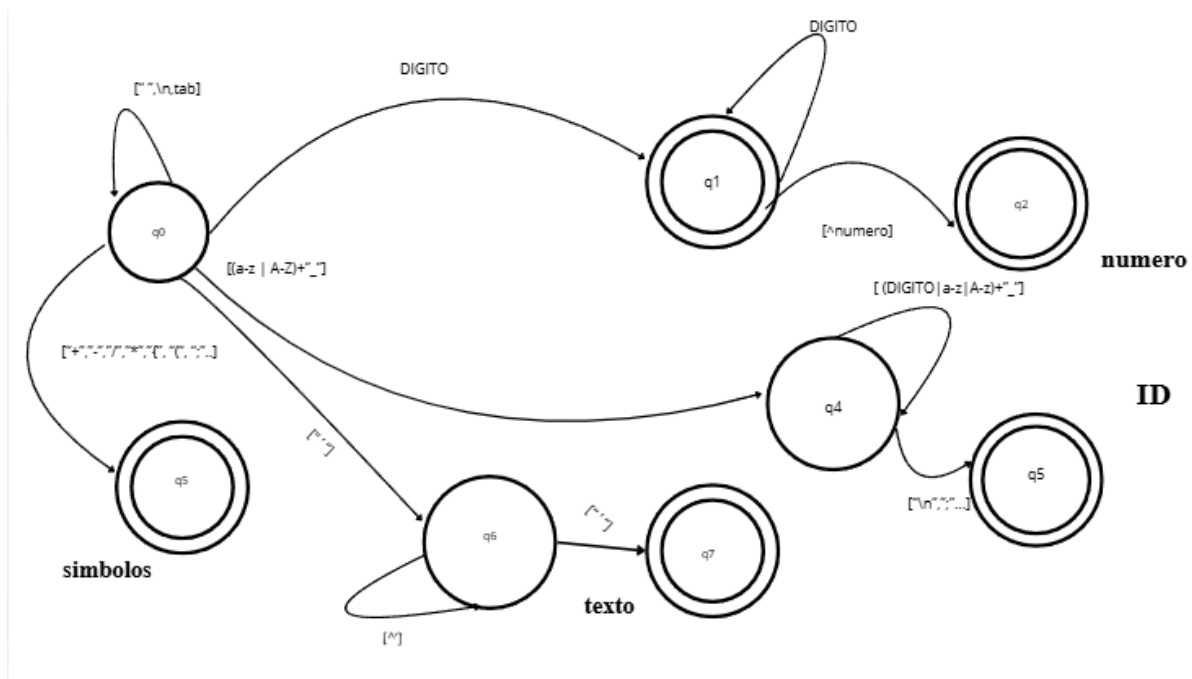
Diseñar e implementar un analizador léxico, sintáctico y semántico utilizando las herramientas Flex y Bison, orientado a la validación de sentencias básicas del lenguaje de manipulación de datos (DML) de MySQL (SELECT, INSERT, UPDATE y DELETE), capaz de procesar archivos de texto por lotes y emitir reportes detallados sobre la estructura y coherencia de las consultas.

Específicos

- Construir el analizador léxico (Flex): Definir las expresiones regulares necesarias para la tokenización del flujo de entrada, reconociendo palabras reservadas de SQL, identificadores, cadenas, números y operadores, ignorando espacios en blanco y llevando un registro exacto del número de línea. .
- Definir la gramática libre de contexto (Bison): Establecer las reglas de producción sintáctica que rigen las operaciones CRUD de SQL para validar que la secuencia de tokens ingresada respete la estructura matemática del lenguaje. .
- Habilitar la lectura de código fuente desde un archivo .txt externo para producir una salida de consola detallada que informe, renglón por renglón, el estado de aceptación o rechazo de cada sentencia, detallando la ubicación y naturaleza (léxica, sintáctica o semántica) de cualquier error encontrado.

4. Desarrollo

Esquema de los AF



Notacion BNF

```
<program> ::= <statements>

<statements> ::= <statements> <statement>
                | <statement>

<statement> ::= <select_stmt> ";"
                | <insert_stmt> ";"
                | <update_stmt> ";"
                | <delete_stmt> ";"

<select_stmt> ::= "SELECT" <select_list> "FROM" <identifier> <where_clause>

<select_list> ::= "*"
                | <column_list>

<column_list> ::= <identifier>
                | <column_list> "," <identifier>

<insert_stmt> ::= "INSERT" "INTO" <identifier> "(" <column_list> ")" "VALUES" "(" <value_list> ")"
                | "INSERT" "INTO" <identifier> "VALUES" "(" <value_list> ")"

<value_list> ::= <value>
                | <value_list> "," <value>

<value> ::= <number>
           | <string>
           | <identifier>

<update_stmt> ::= "UPDATE" <identifier> "SET" <assignment_list> <where_clause>

<assignment_list> ::= <assignment>
                    | <assignment_list> "," <assignment>

<assignment> ::= <identifier> "=" <value>

<delete_stmt> ::= "DELETE" "FROM" <identifier> <where_clause>

<where_clause> ::=
                | "WHERE" <identifier> "=" <value>
```

Código flex

```
%{
#include "parser3.tab.h"
#include <stdio.h>
int line_num = 1; /* Variable para contar los renglones */
%}

%option noyywrap case-insensitive

%%

"SELECT"      { return SELECT; }
"INSERT"      { return INSERT; }
"INTO"        { return INTO; }
"VALUES"      { return VALUES; }
"UPDATE"      { return UPDATE; }
"SET"         { return SET; }
"DELETE"      { return DELETE; }
"FROM"        { return FROM; }
"WHERE"       { return WHERE; }

"*"           { return ASTERISCO; }
"="           { return EQUALS; }
", "          { return COMA; }
"("           { return P_I; }
")"           { return P_D; }
";"           { return PUNTO_COMA; }

[a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFICADOR; }
[0-9]+         { return NUMBER; }
'[^']*'        { return STRING; }

[ \t\r]+      { /* Ignorar espacios, tabulaciones y retornos */ }
\n            { line_num++; }
.             { printf("Línea %d: [ERROR LÉXICO] Carácter '%s' no reconocido.\n", line_num, yytext); }

%%
```

Código Bison

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex();
extern int line_num;
extern char* yytext;
extern FILE* yyin; /* Puntero al archivo de entrada para Flex */
void yyerror(const char *s);
%}

/* Declaración de tokens */
%token SELECT INSERT INTO VALUES UPDATE SET DELETE FROM WHERE
%token ASTERISCO EQUALS COMA P_I P_D PUNTO_COMA
%token IDENTIFICADOR NUMBER STRING

%%

/* Regla principal */
program:
    statements
    ;

statements:
    statements statement
    | statement
    ;

statement:
    select_stmt PUNTO_COMA { printf("Línea %d: [CORRECTA] Sentencia SELECT valida.\n", line_num); }
    | insert_stmt PUNTO_COMA { printf("Línea %d: [CORRECTA] Sentencia INSERT valida.\n", line_num); }
    | update_stmt PUNTO_COMA { printf("Línea %d: [CORRECTA] Sentencia UPDATE valida.\n", line_num); }
    | delete_stmt PUNTO_COMA { printf("Línea %d: [CORRECTA] Sentencia DELETE valida.\n", line_num); }
    | error PUNTO_COMA {
        printf("Línea %d: [ERROR SINTÁCTICO] Falla estructural cerca de '%s'.\n", line_num, yytext);
        yyerrorok; /* Limpia el estado de error de Bison para continuar analizando */
    }
    ;

/* ----- Reglas para SELECT ----- */

select_stmt: SELECT select_list FROM IDENTIFICADOR where_clause;
select_list: ASTERISCO | column_list;
column_list: IDENTIFICADOR | column_list COMA IDENTIFICADOR;

/* ----- Reglas para INSERT ----- */

insert_stmt: INSERT INTO IDENTIFICADOR P_I column_list P_D VALUES P_I value_list P_D
            | INSERT INTO IDENTIFICADOR VALUES P_I value_list P_D;
value_list: value | value_list COMA value;
value: NUMBER | STRING | IDENTIFICADOR;
```

```

/* ----- Reglas para UPDATE ----- */

update_stmt: UPDATE IDENTIFICADOR SET assignment_list where_clause;
assignment_list: assignment | assignment_list COMA assignment;
assignment: IDENTIFICADOR EQUALS value;

/* ----- Reglas para DELETE ----- */

delete_stmt: DELETE FROM IDENTIFICADOR where_clause;

/* ----- Cláusula WHERE ----- */

where_clause:
    /* vacío */
    | WHERE IDENTIFICADOR EQUALS value;

%%

void yyerror(const char *s) {

}

int main(int argc, char **argv) {
    if (argc < 2) {
        printf("Uso incorrecto.\nEjecuta: %s tu_archivo.txt\n", argv[0]);
        return 1;
    }

    /* Abrir el archivo en modo lectura */
    FILE *file = fopen(argv[1], "r");
    if (!file) {
        printf("Error: No se pudo abrir el archivo %s\n", argv[1]);
        return 1;
    }

    /* Redirigir la entrada de Flex (yyin) al archivo en lugar de la consola */
    yyin = file;

    printf("=== Iniciando Análisis del Archivo: %s ===\n", argv[1]);
    yyparse();

    fclose(file);
    printf("=== Análisis Finalizado ===\n");
    return 0;
}

```


Pruebas

El objetivo principal de este programa es procesar un archivo de texto que contiene código fuente y verificar si está correctamente escrito. basándose en un conjunto de reglas gramaticales previamente definidas. Para lograr esto el funcionamiento del sistema se dividió en dos etapas: la identificación de palabras y símbolos ; y la validación de la estructura del código.

1.- El Anallizador Léxico en flex: el trabajo de flex es leer el archivo de texto letra por letra. en esta etapa el analizador no da importancia a si la oracion tiene sentido o no, su unico objetivo es agrupar letras para formar palabras.

Flex funciona usando patrones. cuando lee la secuencia S-E-L-E-C-T, la comparra con su lista de patrones, y si ve que coincide la empaqueta en un token con la etiqueta SELECT y lo manda a la siguiente estación. Por otro lado si encuentra un espacion en blanco, un tabulador o un salto de linea, simplemente lo ignorara y no hara nada.

En este caso cada que el analizador se encuentre un salto de linea sumará 1 a la variable line-num. Esto para llevar un conteo de las lineas y poder escribir la linea exacta en la que se cometió un error.

2.- El Analizador Sintactico en Bison:

Una vez que se reconocen todos los elementos de una instrucción, entra en acción la segunda etapa, donde el programa verifica si el orden en el que se escribieron tiene sentido lógico y obedece a las reglas del lenguaje. Bison recibe la fila de Tokens que le manda flex y revisa si estan ordenados correctamente.

Bison utiliza reglas de producción define una gramatica y con esta información sabe que un SELECT va siempre seguido de columnas y luego FROM y luego nombre de una tabla. Si flex manda algo distinto a esto el orden se marca como roto y lanza un error sintactico.

Resultados que arroja el programa:

Durante la ejecución de esta versión del analizador, el programa genera como salida un reporte en la consola que evalúa el archivo de texto renglón por renglón. Al iniciar, imprime un encabezado indicando el nombre del archivo que está procesando y luego procede a mostrar el resultado de cada línea evaluada. Si una sentencia SQL (ya sea SELECT, INSERT, UPDATE o DELETE) está bien escrita y respeta las reglas de la gramática, el sistema imprime un mensaje de éxito indicando el número de línea y confirmando que la instrucción fue reconocida correctamente. Por el contrario, si la oración tiene símbolos extraños o su estructura está incompleta, el programa no se detiene, sino que arroja una advertencia indicando la línea exacta y la palabra o carácter donde ocurrió el error sintáctico o léxico, continuando con este proceso hasta leer todo el documento y mostrar un mensaje de análisis finalizado.

para la compilación del código se siguieron los siguientes pasos desde la terminal cmd de la computadora.

Se obtienen 3 archivos "lexer3.l" para el analizador léxico "parser3.y" para el analizador sintáctico.

primero se determina la dirección en la que están los archivos .l y .y para su compilación; y desde el cmd se ejecutan los siguientes comandos.

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>flex lexer3.l
```

- esta parte crea el archivo lex.yy.c que contiene un include "parser3.tab.h" para vincularse con el otro analizador sintáctico

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>bison -d parser3.y
```

-esta parte crea los archivos parser.tab.y y parser3.tab.h para su ejecución

-como último paso se genera el .exe de la ejecución del programa con el comando:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>gcc parser3.tab.c lex.yy.c -o "nombre archivo".exe
```

Se ingresa un archivo con sentencias correctas e incorrectas de MySQL, el analizador lo evalúa y determina renglón por renglón si las sentencias son correctas o no, para esto se usa el siguiente comando en el cmd:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>"nombre archivo".exe< prueba3.txt
```

y se obtienen los siguientes resultados:

```
=== Iniciando Análisis del Archivo: prueba3.txt ===
Línea 2: [CORRECTA] Sentencia SELECT válida.
Línea 4: [ERROR SINTÁCTICO] Falla estructural cerca de ';'.
Línea 6: [CORRECTA] Sentencia INSERT válida.
Línea 8: [ERROR SINTÁCTICO] Falla estructural cerca de ';'.
Línea 10: [CORRECTA] Sentencia UPDATE válida.
Línea 12: [ERROR SINTÁCTICO] Falla estructural cerca de ';'.
Línea 14: [CORRECTA] Sentencia DELETE válida.
Línea 16: [ERROR SINTÁCTICO] Falla estructural cerca de ';'.
Línea 18: [CORRECTA] Sentencia SELECT válida.
=== Análisis Finalizado ===
```

5. Conclusiones

La elaboración de esta práctica demostró de manera práctica la distinción fundamental entre la validez de los componentes léxicos y la coherencia estructural de los mismos. El uso de gramáticas de contexto libre, implementadas a través de Yacc y alimentadas por Lex, permite aislar la lógica matemática del lenguaje de programación de la complejidad de leer cadenas de texto. Se concluye que un analizador sintáctico no solo verifica la legalidad del código, sino que también impone un orden jerárquico que es el paso previo e indispensable para la generación de árboles de sintaxis abstracta (AST) y la posterior generación de código máquina.

Referencias Bibliográficas

Carranza Sahagún, D. U. (Coord.). (2024).

Compiladores: Fases de análisis. Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). Lenguajes formales y teoría de autómatas. Editorial Alfaomega. Argentina.

Gutú, O. (2013). Primer curso en teoría de autómatas y lenguajes formales. Pearson Educación. México.

Hopcroft, J. E. Motwani, R. Ullman, J. D. (2007). Introducción a la teoría de automatas, lenguajes y computación. Pearson Educación. México.